SKILLS

Extend Claude with skills

Create, manage, and share skills to extend Claude's capabilities in Claude Code. Includes custom commands and bundled skills.

Skills extend what Claude can do. Create a `SKILL.md` file with instructions, and Claude adds it to its toolkit. Claude uses skills when relevant, or you can invoke one directly with `/skill-name`.

Create a skill when you keep pasting the same instructions, checklist, or multi-step procedure into chat, or when a section of CLAUDE.md has grown into a procedure rather than a fact. Unlike CLAUDE.md content, a skill's body loads only when it's used, so long reference material costs almost nothing until you need it.

For built-in commands like `/help` and `/compact`, and bundled skills like `/debug` and `/code-review`, see the **commands reference**.

Custom commands have been merged into skills. A file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way. Your existing `.claude/commands/` files keep working. Skills add optional features: a directory for supporting files, frontmatter to **control whether you or Claude invokes them**, and the ability for Claude to load them automatically when relevant.

Claude Code skills follow the **Agent Skills** open standard, which works across multiple AI tools. Claude Code extends the standard with additional features like **invocation control**, **subagent execution**, and **dynamic context injection**.

Bundled skills

Claude Code includes a set of bundled skills that are available in every session unless disabled with the `disableBundledSkills` setting, including `/code-review`, `/batch`, `/debug`, `/loop`, and `/claude-api`. Unlike most built-in commands, which execute fixed logic directly, bundled skills are prompt-based: they give Claude detailed instructions and let it orchestrate the work using its tools. You invoke them the same way as any other skill, by typing `/` followed by the skill name.

Bundled skills are listed alongside built-in commands in the **commands reference**, marked **Skill** in the

Purpose column.

Run and verify your app

Three bundled skills work together to launch your app and confirm changes against the running app instead of just tests:

Skill	Purpose
`/run`	Launch and drive your app to see a change working
`/verify`	Build and run your app to confirm a code change does what it should, without falling back to tests or type checks
`/run-skill-generator`	Teach `/run` and `/verify` how to build and launch your project

All three skills require Claude Code v2.1.145 or later.

`/run` and `/verify` work without setup. They infer the launch from your project type (CLI, server, TUI, browser-driven) and from what's in your README, `package.json`, or `Makefile`. That inference gets unreliable for projects that need anything beyond a standard launch: a database, an env file, a graphical session, a multi-step build.

`/run-skill-generator` records the recipe instead. It gets your app running from a clean environment, captures what worked (the install commands, the env vars, the launch script), and commits it as a per-project skill at `.claude/skills/run-<name>/`. After that, `/run`, `/verify`, and any other agent in the repo follow the recorded recipe instead of rediscovering it. Run `/run-skill-generator` once per project, and again if the build or launch process changes.

Getting started

Create your first skill

This example creates a skill that summarizes the uncommitted changes in your git repository and flags anything risky. It pulls the live diff into the prompt before Claude reads it, so the response is grounded in your actual working tree rather than what Claude can guess from open files. Claude loads the skill automatically when you ask about your changes, or you can invoke it directly with `/summarize-changes`.

1 Create the skill directory

Create a directory for the skill in your personal skills folder. Personal skills are available across all your projects.

```
mkdir -p ~/.claude/skills/summarize-changes
```

2 Write SKILL.md

Every skill needs a `SKILL.md` file with two parts: YAML frontmatter between `---` markers that tells Claude when to use the skill, and markdown content with the instructions Claude follows when the skill runs. The directory name becomes the command you type, and the `description` helps Claude decide when to load the skill automatically.

Save this to `~/.claude/skills/summarize-changes/SKILL.md`:

```
---
description: Summarizes uncommitted changes and flags
anything risky. Use when the user asks what changed, wants a
commit message, or asks to review their diff.
---

## Current changes

!`git diff HEAD`

## Instructions

Summarize the changes above in two or three bullet points,
then list any risks you notice such as missing error
handling, hardcoded values, or tests that need updating. If
the diff is empty, say there are no uncommitted changes.
```

The `!`git diff HEAD`` line uses **dynamic context injection**: Claude Code runs the command and replaces the line with its output before Claude sees the skill content, so the instructions arrive with the current diff already inlined.

3 Test the skill

Open a git project, make a small edit to any file, and start Claude Code by running `claude`. You can test the skill two ways.

Let Claude invoke it automatically by asking something that matches the description:

```
What did I change?
```

Or invoke it directly with the skill name:

```
/summarize-changes
```

Either way, Claude should respond with a short summary of your edit and a list of risks.

Where skills live

Where you store a skill determines who can use it:

Location	Path	Applies to
Enterprise	See **managed settings**	All users in your organization
Personal	`~/.claude/skills/<skill-name>/SKILL.md`	All your projects
Project	`.claude/skills/<skill-name>/SKILL.md`	This project only
Plugin	`<plugin>/skills/<skill-name>/SKILL.md`	Where plugin is enabled

When skills share the same name across levels, enterprise overrides personal, and personal overrides project. A skill at any of these levels also overrides a bundled skill with the same name. For example, a `code-review` skill in your project's `.claude/skills/` replaces the bundled `/code-review`. Plugin skills use a `plugin-name:skill-name` namespace, so they cannot conflict with other levels. If you have files in `.claude/commands/`, those work the same way, but if a skill and a command share the same name, the skill takes precedence.

Skills also load from nested `.claude/skills/` directories below your working directory. When Claude reads or edits a file in a subdirectory, skills from that subdirectory's `.claude/skills/` become available. This lets a monorepo package provide its own skills that apply when working on that package, even if the session started at the repo root.

If a nested skill shares a name with another skill, both stay available. For example, with a `deploy` skill at the project root and another in `apps/web/.claude/skills/`:

- The nested one appears under a directory-qualified name, `apps/web:deploy`.
- Its description says which directory it applies to.
- Claude picks the variant that matches the files it is working on.

Typing `/deploy` runs the project-root skill. Type the qualified name `/apps/web:deploy` to run the nested variant explicitly.

> Add a `.claude-plugin/plugin.json` to a skill folder and it loads as a **plugin** named `<name>@skills-dir`, so it can bundle agents, hooks, and MCP servers. In a project's `.claude/skills/`, this requires accepting the workspace trust dialog first.

Live change detection

Claude Code watches skill directories for file changes. Adding, editing, or removing a skill under `~/.claude/skills/`, the project `.claude/skills/`, or a `.claude/skills/` inside an `--add-dir` directory takes effect within the current session without restarting. Creating a top-level skills directory that did not exist when the session started requires restarting Claude Code so the new directory can be watched.

> Live change detection covers `SKILL.md` text only. For a skill folder that is also a **plugin**, changes to `hooks/`, `.mcp.json`, `agents/`, and `output-styles/` need `/reload-plugins` to take effect.

Automatic discovery from parent and nested directories

Project skills load from `.claude/skills/` in your starting directory and in every parent directory up to the repository root, so starting Claude in a subdirectory still picks up skills defined at the root. When you work with files in subdirectories below your starting directory, Claude Code also discovers skills from nested `.claude/skills/` directories on demand. For example, if you're editing a file in `packages/frontend/`, Claude Code also looks for skills in `packages/frontend/.claude/skills/`. This supports monorepo setups where packages have their own skills.

Each skill is a directory with `SKILL.md` as the entrypoint:

```
my-skill/
├── SKILL.md           # Main instructions (required)
├── template.md        # Template for Claude to fill in
├── examples/
│   └── sample.md      # Example output showing expected format
└── scripts/
    └── validate.sh    # Script Claude can execute
```

The `SKILL.md` contains the main instructions and is required. Other files are optional and let you build more powerful skills: templates for Claude to fill in, example outputs showing the expected format, scripts Claude can execute, or detailed reference documentation. Reference these files from your `SKILL.md` so Claude knows what they contain and when to load them. See **Add supporting files** for more details.

> Files in `.claude/commands/` still work and support the same **frontmatter**. Skills are recommended since they support additional features like supporting files.

Skills from additional directories

The `--add-dir` flag and `/add-dir` command **grant file access** rather than configuration discovery, but skills are an exception: `.claude/skills/` within an added directory is loaded automatically. This exception applies only to `--add-dir` and `/add-dir`. The `permissions.additionalDirectories` setting in `settings.json` grants file access only and does not load skills. See **Live change detection** for how edits are picked up during a session.

Other `.claude/` configuration such as commands and output styles is not loaded from additional directories. See the **exceptions table** for the complete list of what is and isn't loaded, and the recommended ways to share configuration across projects.

> CLAUDE.md files from `--add-dir` directories are not loaded by default. To load them, set `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1`. See **Load from additional directories**.

Configure skills

Skills are configured through YAML frontmatter at the top of `SKILL.md` and the markdown content that follows.

Types of skill content

Skill files can contain any instructions, but thinking about how you want to invoke them helps guide what to include:

Reference content adds knowledge Claude applies to your current work. Conventions, patterns, style guides, domain knowledge. This content runs inline so Claude can use it alongside your conversation context.

```
---
name: api-conventions
description: API design patterns for this codebase
---

When writing API endpoints:
- Use RESTful naming conventions
- Return consistent error formats
- Include request validation
```

Task content gives Claude step-by-step instructions for a specific action, like deployments, commits, or code generation. These are often actions you want to invoke directly with `/skill-name` rather than letting Claude decide when to run them. Add `disable-model-invocation: true` to prevent Claude from triggering it automatically.

```
---
name: deploy
description: Deploy the application to production
context: fork
disable-model-invocation: true
---

Deploy the application:
1. Run the test suite
2. Build the application
3. Push to the deployment target
```

Your `SKILL.md` can contain anything, but thinking through how you want the skill invoked (by you, by Claude, or both) and where you want it to run (inline or in a subagent) helps guide what to include. For complex skills, you can also **add supporting files** to keep the main skill focused.

Keep the body itself concise. Once a skill loads, its content **stays in context across turns**, so every line is a recurring token cost. State what to do rather than narrating how or why, and apply the same conciseness test you would for **CLAUDE.md content**.

Frontmatter reference

Beyond the markdown content, you can configure skill behavior using YAML frontmatter fields between `---` markers at the top of your `SKILL.md` file:

```
---
name: my-skill
description: What this skill does
disable-model-invocation: true
allowed-tools: Read Grep
---

Your skill instructions here...
```

All fields are optional. Only `description` is recommended so Claude knows when to use the skill.

Field	Required	Description
`name`	No	Display name shown in skill listings. Defaults to the directory name. See **How a skill gets its command name** for how this differs from the name you type to invoke the skill.
`description`	Recommended	What the skill does and when to use it. Claude uses this to decide when to apply the skill. If omitted, uses the first paragraph of markdown content. Put the key use case first: the combined `description` and `when_to_use` text is truncated at 1,536 characters in the skill listing to reduce context usage.
`when_to_use`	No	Additional context for when Claude should invoke the skill, such as trigger phrases or example requests. Appended to `description` in the skill listing and counts toward the 1,536-character cap.
`argument-hint`	No	Hint shown during autocomplete to indicate expected arguments. Example: `[issue-number]` or `[filename] [format]`.
`arguments`	No	Named positional arguments for `$name` **substitution** in the skill content. Accepts a space-separated string or a YAML list. Names map to argument positions in order.
`disable-model-invocation`	No	Set to `true` to prevent Claude from automatically loading this skill. Use for workflows you want to trigger manually with `/name`. Also prevents the skill from being **preloaded into subagents**.

Field	Required	Description
		Default: `false`.
`user-invocable`	No	Set to `false` to hide from the `/` menu. Use for background knowledge users shouldn't invoke directly. Default: `true`.
`allowed-tools`	No	Tools Claude can use without asking permission when this skill is active. Accepts a space- or comma-separated string, or a YAML list.
`disallowed-tools`	No	Tools removed from Claude's available pool while this skill is active. Use for autonomous skills that should never call certain tools, such as `AskUserQuestion` for a background loop. Accepts a space- or comma-separated string, or a YAML list. The restriction clears when you send your next message.
`model`	No	Model to use when this skill is active. The override applies for the rest of the current turn and is not saved to settings; the session model resumes on your next prompt. Accepts the same values as `/model`, or `inherit` to keep the active model. A value excluded by your organization's `availableModels` allowlist is not used and the session keeps its current model.
`effort`	No	**Effort level** when this skill is active. Overrides the session effort level. Default: inherits from session. Options: `low`, `medium`, `high`, `xhigh`, `max`; available levels depend on the model.
`context`	No	Set to `fork` to run in a forked subagent context.
`agent`	No	Which subagent type to use when `context: fork` is set.
`hooks`	No	Hooks scoped to this skill's lifecycle. See **Hooks in skills and agents** for configuration format.
`paths`	No	Glob patterns that limit when this skill is activated. Accepts a comma-separated string or a YAML list. When set, Claude loads the skill automatically only when working with files matching the patterns. Uses the same format as **path-specific rules**.
`shell`	No	Shell to use for `!`command`` and ```! blocks in this skill. Accepts `bash` (default) or `powershell`. Setting `powershell` runs inline shell commands via PowerShell on Windows. Requires `CLAUDE_CODE_USE_POWERSHELL_TOOL=1`.

How a skill gets its command name

The command you type to invoke a skill comes from where the skill file lives. The frontmatter `name` field sets the display label shown in skill listings and, except for a plugin-root `SKILL.md`, does not change what you type after `/`.

The table below shows where the command name comes from for each layout:

Skill location	Command name source	Example
Skill directory under `~/.claude/skills/` or `.claude/skills/`	Directory name	`.claude/skills/deploy-staging/SKILL.md` → `/deploy-staging`
Nested `.claude/skills/` directory, when the name clashes with another skill	Subdirectory path relative to the working directory, then the skill directory name	`apps/web/.claude/skills/deploy/SKILL.md` → `/apps/web:deploy`
File under `.claude/commands/`	File name without extension	`.claude/commands/deploy.md` → `/deploy`
Plugin `skills/` subdirectory	Directory name, namespaced by plugin	`my-plugin/skills/review/SKILL.md` → `/my-plugin:review`
Plugin root `SKILL.md`	Frontmatter `name`, with the plugin directory name as a fallback	`my-plugin/SKILL.md` with `name: review` → `/my-plugin:review`. See **Path behavior rules**

The plugin-root case is the one place where `name` does set the command name, because there is no skill directory to take it from. If `name` is not set in the frontmatter, the plugin's directory name is used instead.

Available string substitutions

Skills support string substitution for dynamic values in the skill content:

Variable	Description
`$ARGUMENTS`	All arguments passed when invoking the skill. If `$ARGUMENTS` is not present in the content, arguments are appended as `ARGUMENTS: <value>`.
`$ARGUMENTS[N]`	Access a specific argument by 0-based index, such as `$ARGUMENTS[0]` for the first argument.
`$N`	Shorthand for `$ARGUMENTS[N]`, such as `$0` for the first argument or `$1` for the second.
`$name`	Named argument declared in the `arguments` frontmatter list. Names map to positions in order, so with `arguments: [issue, branch]` the placeholder `$issue` expands to the first argument and `$branch` to the second.
`${CLAUDE_SESSION_ID}`	The current session ID. Useful for logging, creating session-specific files, or correlating skill output with sessions.
`${CLAUDE_EFFORT}`	The current effort level: `low`, `medium`, `high`, `xhigh`, or `max`. Ultracode is not a distinct level and reports as `xhigh`. Use this to adapt skill instructions to the active effort setting.

Variable	Description
`${CLAUDE_SKILL_DIR}`	The directory containing the skill's `SKILL.md` file. For plugin skills, this is the skill's subdirectory within the plugin, not the plugin root. Use this in bash injection commands to reference scripts or files bundled with the skill, regardless of the current working directory.

Indexed arguments use shell-style quoting, so wrap multi-word values in quotes to pass them as a single argument. For example, `/my-skill "hello world" second` makes `$0` expand to `hello world` and `$1` to `second`. The `$ARGUMENTS` placeholder always expands to the full argument string as typed.

To include a literal `$` before a digit, `ARGUMENTS`, or a declared argument name, such as `$1.00` in prose, escape it with a backslash: `\$1.00`. A backslash before any other `$` is left unchanged. Only a single backslash directly before the token escapes it. A doubled backslash such as `\\$1` leaves both backslashes in place, and `$1` still expands to the argument value.

Example using substitutions:

```
---
name: session-logger
description: Log activity for this session
---

Log the following to logs/${CLAUDE_SESSION_ID}.log:

$ARGUMENTS
```

Add supporting files

Skills can include multiple files in their directory. This keeps `SKILL.md` focused on the essentials while letting Claude access detailed reference material only when needed. Large reference docs, API specifications, or example collections don't need to load into context every time the skill runs.

```
my-skill/
├── SKILL.md (required - overview and navigation)
├── reference.md (detailed API docs - loaded when needed)
├── examples.md (usage examples - loaded when needed)
└── scripts/
    └── helper.py (utility script - executed, not loaded)
```

Reference supporting files from `SKILL.md` so Claude knows what each file contains and when to load it:

```
## Additional resources

- For complete API details, see [reference.md](reference.md)
- For usage examples, see [examples.md](examples.md)
```

Keep `SKILL.md` under 500 lines. Move detailed reference material to separate files.

Control who invokes a skill

By default, both you and Claude can invoke any skill. You can type `/skill-name` to invoke it directly, and Claude can load it automatically when relevant to your conversation. Two frontmatter fields let you restrict this:

- `disable-model-invocation: true`: Only you can invoke the skill. Use this for workflows with side effects or that you want to control timing, like `/commit`, `/deploy`, or `/send-slack-message`. You don't want Claude deciding to deploy because your code looks ready.
- `user-invocable: false`: Only Claude can invoke the skill. Use this for background knowledge that isn't actionable as a command. A `legacy-system-context` skill explains how an old system works. Claude should know this when relevant, but `/legacy-system-context` isn't a meaningful action for users to take.

This example creates a deploy skill that only you can trigger. The `disable-model-invocation: true` field prevents Claude from running it automatically:

```
---
name: deploy
description: Deploy the application to production
disable-model-invocation: true
---

Deploy $ARGUMENTS to production:

1. Run the test suite
2. Build the application
3. Push to the deployment target
4. Verify the deployment succeeded
```

Here's how the two fields affect invocation and context loading:

Frontmatter	You can invoke	Claude can invoke	When loaded into context
(default)	Yes	Yes	Description always in context, full skill loads when invoked
`disable-model-invocation: true`	Yes	No	Description not in context, full skill loads when you invoke
`user-invocable: false`	No	Yes	Description always in context, full skill loads when invoked

> In a regular session, skill descriptions are loaded into context so Claude knows what's available, but full skill content only loads when invoked. **Subagents with preloaded skills** work differently: the full skill content is injected at startup.

Skill content lifecycle

When you or Claude invoke a skill, the rendered `SKILL.md` content enters the conversation as a single message and stays there for the rest of the session. Claude Code does not re-read the skill file on later turns, so write guidance that should apply throughout a task as standing instructions rather than one-time steps.

Auto-compaction carries invoked skills forward within a token budget. When the conversation is summarized to free context, Claude Code re-attaches the most recent invocation of each skill after the summary, keeping the first 5,000 tokens of each. Re-attached skills share a combined budget of 25,000 tokens. Claude Code fills this budget starting from the most recently invoked skill, so older

skills can be dropped entirely after compaction if you have invoked many in one session.

If a skill seems to stop influencing behavior after the first response, the content is usually still present and the model is choosing other tools or approaches. Strengthen the skill's `description` and instructions so the model keeps preferring it, or use **hooks** to enforce behavior deterministically. If the skill is large or you invoked several others after it, re-invoke it after compaction to restore the full content.

Pre-approve tools for a skill

The `allowed-tools` field grants permission for the listed tools while the skill is active, so Claude can use them without prompting you for approval. It does not restrict which tools are available: every tool remains callable, and your **permission settings** still govern tools that are not listed.

For skills checked into a project's `.claude/skills/` directory, `allowed-tools` takes effect after you accept the workspace trust dialog for that folder, the same as permission rules in `.claude/settings.json`. Review project skills before trusting a repository, since a skill can grant itself broad tool access.

This skill lets Claude run git commands without per-use approval whenever you invoke it:

```
---
name: commit
description: Stage and commit the current changes
disable-model-invocation: true
allowed-tools: Bash(git add *) Bash(git commit *) Bash(git status
*)
---
```

To remove tools from Claude's available pool while a skill is active, list them in `disallowed-tools` in the skill's frontmatter. The restriction clears when you send your next message. To block tools across all skills and prompts, add deny rules in your **permission settings**.

Pass arguments to skills

Both you and Claude can pass arguments when invoking a skill. Arguments are available via the `$ARGUMENTS` placeholder.

This skill fixes a GitHub issue by number. The `$ARGUMENTS` placeholder gets replaced with whatever

follows the skill name:

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---

Fix GitHub issue $ARGUMENTS following our coding standards.

1. Read the issue description
2. Understand the requirements
3. Implement the fix
4. Write tests
5. Create a commit
```

When you run `/fix-issue 123`, Claude receives "Fix GitHub issue 123 following our coding standards..."

If you invoke a skill with arguments but the skill doesn't include `$ARGUMENTS`, Claude Code appends `ARGUMENTS: <your input>` to the end of the skill content so Claude still sees what you typed.

To access individual arguments by position, use `$ARGUMENTS[N]` or the shorter `$N`:

```
---
name: migrate-component
description: Migrate a component from one framework to another
---

Migrate the $ARGUMENTS[0] component from $ARGUMENTS[1] to
$ARGUMENTS[2].
Preserve all existing behavior and tests.
```

Running `/migrate-component SearchBar React Vue` replaces `$ARGUMENTS[0]` with `SearchBar`, `$ARGUMENTS[1]` with `React`, and `$ARGUMENTS[2]` with `Vue`. The same skill using the `$N` shorthand:

```
---
name: migrate-component
description: Migrate a component from one framework to another
---

Migrate the $0 component from $1 to $2.
Preserve all existing behavior and tests.
```

Advanced patterns

Inject dynamic context

The `!`<command>`` syntax runs shell commands before the skill content is sent to Claude. The command output replaces the placeholder, so Claude receives actual data, not the command itself.

This skill summarizes a pull request by fetching live PR data with the GitHub CLI. The `!`gh pr diff`` and other commands run first, and their output gets inserted into the prompt:

```
---
name: pr-summary
description: Summarize changes in a pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---

## Pull request context
- PR diff: !`gh pr diff`
- PR comments: !`gh pr view --comments`
- Changed files: !`gh pr diff --name-only`

## Your task
Summarize this pull request...
```

When this skill runs:

1. Each `!`<command>`` executes immediately (before Claude sees anything)
2. The output replaces the placeholder in the skill content
3. Claude receives the fully-rendered prompt with actual PR data

This is preprocessing, not something Claude executes. Claude only sees the final result.

Substitution runs once over the original file. Command output is inserted as plain text and is not re-scanned for further `!`<command>`` placeholders, so a command cannot emit a placeholder for a later pass to expand.

The inline form is only recognized when `!` appears at the start of a line or immediately after whitespace. If `!` follows another character, as in `KEY=!`cmd``, the placeholder is left as literal text and the command does not run.

For multi-line commands, use a fenced code block opened with ```` ```! ```` instead of the inline form:

````
## Environment
```!
node --version
npm --version
git status --short
```
````

To disable this behavior for skills and custom commands from user, project, plugin, or **additional-directory** sources, set `"disableSkillShellExecution": true` in **settings**. Each command is replaced with `[shell command execution disabled by policy]` instead of being run. Bundled and managed skills are not affected. This setting is most useful in **managed settings**, where users cannot override it.

> To request deeper reasoning when a skill runs, include `ultrathink` anywhere in the skill content. See **Use ultrathink for one-off deep reasoning**.

Run skills in a subagent

Add `context: fork` to your frontmatter when you want a skill to run in isolation. The skill content becomes the prompt that drives the subagent. It won't have access to your conversation history.

> ⚠ `context: fork` only makes sense for skills with explicit instructions. If your skill contains guidelines like "use these API conventions" without a task, the subagent receives the guidelines but no actionable prompt, and returns without meaningful output.

Skills and **subagents** work together in two directions:

Approach	System prompt	Task	Also loads
Skill with `context: fork`	From agent type	SKILL.md content	CLAUDE.md, except when the agent is Explore or Plan
Subagent with `skills` field	Subagent's markdown body	Claude's delegation message	Preloaded skills + CLAUDE.md

With `context: fork`, you write the task in your skill and pick an agent type to execute it. The built-in Explore and Plan agents **skip CLAUDE.md and git status** to keep their context small, so a forked skill using `agent: Explore` sees only the SKILL.md content and the agent's own system prompt. For the inverse, where you define a custom subagent that uses skills as reference material, see **Subagents**.

Example: Research skill using Explore agent

This skill runs research in a forked Explore agent. The skill content becomes the task, and the agent provides read-only tools optimized for codebase exploration:

```
---
name: deep-research
description: Research a topic thoroughly
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:

1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with specific file references
```

When this skill runs:

1. A new isolated context is created
2. The subagent receives the skill content as its prompt ("Research $ARGUMENTS thoroughly…")
3. The `agent` field determines the execution environment (model, tools, and permissions)
4. Results are summarized and returned to your main conversation

The `agent` field specifies which subagent configuration to use. Options include built-in agents

(`Explore` , `Plan` , `general-purpose`) or any custom subagent from `.claude/agents/` . If omitted, uses `general-purpose` .

Restrict Claude's skill access

By default, Claude can invoke any skill that doesn't have `disable-model-invocation: true` set. Skills that define `allowed-tools` grant Claude access to those tools without per-use approval when the skill is active. Your **permission settings** still govern baseline approval behavior for all other tools. A few built-in commands are also available through the Skill tool, including `/init` , `/review` , and `/security-review` . Other built-in commands such as `/compact` are not.

Three ways to control which skills Claude can invoke:

Disable all skills by denying the Skill tool in `/permissions` :

```
# Add to deny rules:
Skill
```

Allow or deny specific skills using **permission rules**:

```
# Allow only specific skills
Skill(commit)
Skill(review-pr *)

# Deny specific skills
Skill(deploy *)
```

Permission syntax: `Skill(name)` for exact match, `Skill(name *)` for prefix match with any arguments.

Hide individual skills by adding `disable-model-invocation: true` to their frontmatter. This removes the skill from Claude's context entirely.

> The `user-invocable` field only controls menu visibility, not Skill tool access. Use `disable-model-invocation: true` to block programmatic invocation.

Override skill visibility from settings

The `skillOverrides` setting controls skill visibility from your **settings** instead of the skill's own frontmatter. Use it for skills whose SKILL.md you don't want to edit, such as ones checked into a shared project repo or provided by an MCP server. The `/skills` menu writes it for you: highlight a skill and press `Space` to cycle states, then `Enter` to save to `.claude/settings.local.json`.

Each key is a skill name and each value is one of four states:

Value	Listed to Claude	In `/` menu
`"on"`	Name and description	Yes
`"name-only"`	Name only	Yes
`"user-invocable-only"`	Hidden	Yes
`"off"`	Hidden	Hidden

A skill that is absent from `skillOverrides` is treated as `"on"`. The example below collapses one skill to its name and turns another off entirely:

```
{
  "skillOverrides": {
    "legacy-context": "name-only",
    "deploy": "off"
  }
}
```

Plugin skills are not affected by `skillOverrides`. Manage those through `/plugin` instead.

Evaluate and iterate on a skill

Seeing a skill trigger tells you Claude found it, not that it did what you intended. To know a skill is working, measure two things separately: whether Claude invokes it on the prompts it should, and whether the output matches what you expect when it does.

The check for both is a baseline comparison. Collect a few realistic prompts, run each one in a fresh session with the skill available and again with it **disabled**, and compare the results. A fresh session matters because leftover context from authoring the skill will mask gaps in the written instructions.

Run evals with skill-creator

The `skill-creator` **plugin** automates the comparison loop inside Claude Code. Install it from the official marketplace:

```
/plugin install skill-creator@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

After installing, run `/reload-plugins` to make the plugin's skills available in the current session. Then ask Claude to evaluate an existing skill, for example `evaluate my summarize-changes skill with skill-creator`. The plugin walks you through writing test cases and runs the loop:

- **Test cases**: stores prompts, input files, and expected behavior in `evals/evals.json` inside the skill directory
- **Isolated runs**: spawns a **subagent** per test case so each run starts with a clean context, and records token count and duration
- **Grading**: checks each assertion against the output and writes pass or fail with evidence to `grading.json`
- **Benchmark**: aggregates pass rate, time, and tokens for with-skill versus without-skill into `benchmark.json` so you can compare the pass-rate improvement against the token and time overhead
- **Version comparison**: runs a blind A/B between two versions of the skill so you can confirm an edit is an improvement before committing it
- **Description tuning**: generates should-trigger and should-not-trigger prompts, measures the hit rate, and proposes description edits when the skill activates on the wrong requests
- **Review viewer**: opens an HTML report where you inspect each output and record qualitative feedback that the next iteration reads

For the eval file format and the full iteration workflow, see **Evaluating skill output quality** on agentskills.io. For background on the benchmark and comparison modes, see the **skill-creator announcement**.

Share skills

Skills can be distributed at different scopes depending on your audience:

- **Project skills**: Commit `.claude/skills/` to version control
- **Plugins**: Create a `skills/` directory in your **plugin**
- **Managed**: Deploy organization-wide through **managed settings**

Generate visual output

Skills can bundle and run scripts in any language, giving Claude capabilities beyond what's possible in a single prompt. One powerful pattern is generating visual output: interactive HTML files that open in your browser for exploring data, debugging, or creating reports.

This example creates a codebase explorer: an interactive tree view where you can expand and collapse directories, see file sizes at a glance, and identify file types by color.

Create the Skill directory:

```
mkdir -p ~/.claude/skills/codebase-visualizer/scripts
```

Save this to `~/.claude/skills/codebase-visualizer/SKILL.md`. The description tells Claude when to activate this Skill, and the instructions tell Claude to run the bundled script. The script path uses `${CLAUDE_SKILL_DIR}` so it resolves correctly whether the skill is installed at the personal, project, or plugin level:

````
---
name: codebase-visualizer
description: Generate an interactive collapsible tree
visualization of your codebase. Use when exploring a new repo,
understanding project structure, or identifying large files.
allowed-tools: Bash(python3 *)
---

# Codebase Visualizer

Generate an interactive HTML tree view that shows your project's
file structure with collapsible directories.

## Usage

Run the visualization script from your project root:

```bash
python3 ${CLAUDE_SKILL_DIR}/scripts/visualize.py .
```

This creates `codebase-map.html` in the current directory and
opens it in your default browser.

## What the visualization shows

- **Collapsible directories**: Click folders to expand/collapse
- **File sizes**: Displayed next to each file
- **Colors**: Different colors for different file types
- **Directory totals**: Shows aggregate size of each folder
````

Save this to `~/.claude/skills/codebase-visualizer/scripts/visualize.py`. This script scans a directory tree and generates a self-contained HTML file with:

- A **summary sidebar** showing file count, directory count, total size, and number of file types
- A **bar chart** breaking down the codebase by file type (top 8 by size)
- A **collapsible tree** where you can expand and collapse directories, with color-coded file type indicators

The script requires Python 3 but uses only built-in libraries, so there are no packages to install:

```
#!/usr/bin/env python3
"""Generate an interactive collapsible tree visualization of a
codebase."""

import json
import sys
import webbrowser
```

··· See all 133 lines

To test, open Claude Code in any project and ask “Visualize this codebase.” Claude runs the script, generates `codebase-map.html`, and opens it in your browser.

This pattern works for any visual output: dependency graphs, test coverage reports, API documentation, or database schema visualizations. The bundled script does the work while Claude handles orchestration.

Troubleshooting

Skill not triggering

If Claude doesn’t use your skill when expected:

1. Check the description includes keywords users would naturally say
2. Verify the skill appears in `What skills are available?`
3. Try rephrasing your request to match the description more closely
4. Invoke it directly with `/skill-name` if the skill is user-invocable

If the frontmatter YAML is malformed, Claude Code loads the skill body with empty metadata, so `/skill-name` still works but Claude has no `description` to match against. Run with `--debug` to see the parse error.

Skill triggers too often

If Claude uses your skill when you don’t want it:

1. Make the description more specific
2. Add `disable-model-invocation: true` if you only want manual invocation

Skill descriptions are cut short

Skill descriptions are loaded into context so Claude knows what's available. All skill names are always included, but if you have many skills, descriptions are shortened to fit the character budget, which can strip the keywords Claude needs to match your request. The budget scales at 1% of the model's context window. When it overflows, descriptions for the skills you invoke least are dropped first, so the skills you actually use keep their full text. Run `/doctor` to see how many skill descriptions are being shortened or dropped and which skills are affected.

To raise the budget, set the `skillListingBudgetFraction` setting (e.g. `0.02` = 2%) or the `SLASH_COMMAND_TOOL_CHAR_BUDGET` environment variable to a fixed character count. To free budget for other skills, set low-priority entries to `"name-only"` in `skillOverrides` so they list without a description. You can also trim the `description` and `when_to_use` text at the source: put the key use case first, since each entry's combined text is capped at 1,536 characters regardless of budget. The cap is configurable with `maxSkillDescriptionChars`.

Related resources

- **Debug your configuration**: diagnose why a skill isn't appearing or triggering
- **Evaluating skill output quality**: the eval file format and iteration workflow on agentskills.io
- **Skill authoring best practices**: writing guidance that applies across Claude products
- **Subagents**: delegate tasks to specialized agents
- **Plugins**: package and distribute skills with other extensions
- **Hooks**: automate workflows around tool events
- **Memory**: manage CLAUDE.md files for persistent context
- **Commands**: reference for built-in commands and bundled skills
- **Permissions**: control tool and skill access
- **Claude Tag skills**: project skills committed to a repo also load when that repo is used in a Claude Tag channel